



University
of Basel

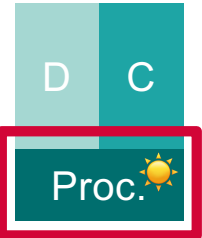
Chapter 5.2: Parallel Architectures and Programming Models – Memory Foundations of Distributed Systems

Fall Semester 2025

Prof. Dr. Florina Ciorba

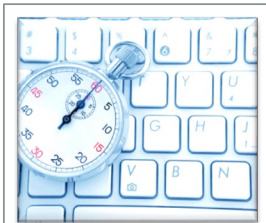


Addressed in The **Processing** Lecture Series



- Goals

Chapter 5.2:
Parallel architectures & models



Performance

```
00111101001010101
01001010100010101
01010ERROR011010
10100010100110001
01100010100100111
10100010100110001
```

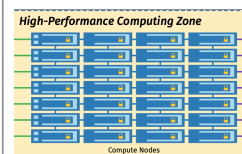
Resiliency



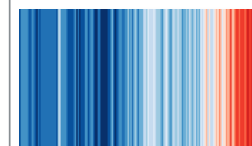
Extreme scalability



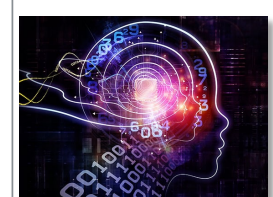
Energy



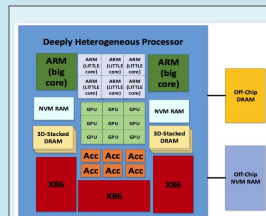
Security



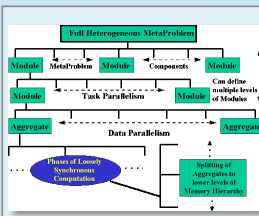
Sustainability



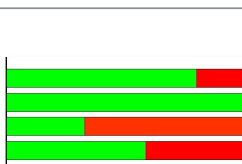
Convergence
HPC+Data+AI



Heterogeneity



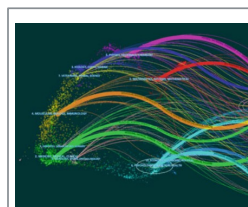
Multilevel parallelism



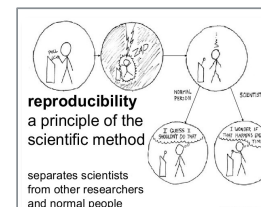
Load imbalance



Programming



Visualization



Reproducibility

Overview of Chapter 5.2: Parallel Architectures & Programming Models

a	Memory Architectures
b	Parallel Programming Models
c	Network Topologies

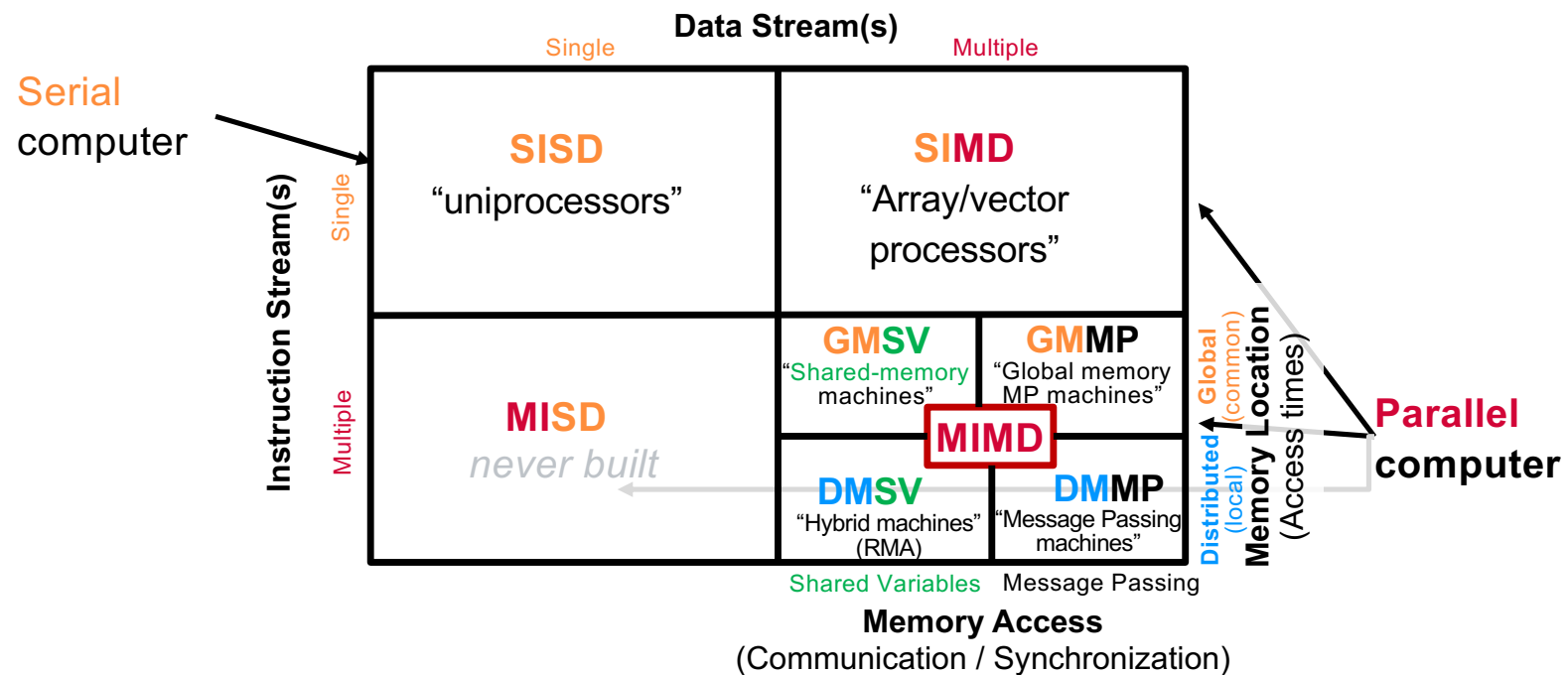
Learning objectives

Understand parallel computer architectures, including memory organization and network interconnections.

Understand their role in defining parallel programming models.

Q: How to Program Such Parallel Computers?

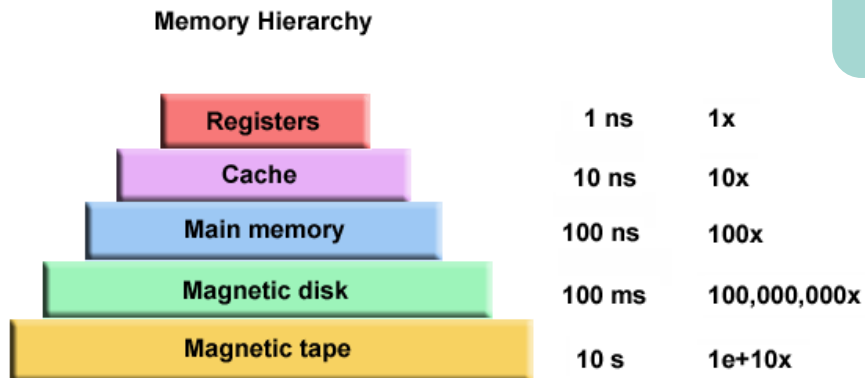
Motivating Interest in Memory Architectures of Parallel Machines



Parallel Computers and Their Programming (Models)

As Easy as A, B, C

A. Memory architectures



What the machine offers
w.r.t. memory

B. Programming models

C. Network topologies

A. Memory Architectures

Note on Uniform Memory Access (UMA)

Uniform Memory Access (UMA) architectures

- There is a single **unified memory address space (UAS)** visible to all CPUs
 - Access to this **memory address space** is via **LOAD** and **STORE** instructions
 - Access cost to this **memory address space** is **equal** (uniform) to the access cost to any local memory (access to memory word in local memory/cache ~**2-10 nsec**) ⇒ **UMA**
 - Hardware ensure cache coherency across processors
- ⇒ Access cost is **uniform** and generally invisible to programmers

Non-uniform Memory Access (NUMA) architectures

- There is a single **memory address space** that may be partially visible to all CPUs (dep. on hw. topology)
- Access to this **memory address space** is via
 - **LOAD/STORE** (shared-memory operations where **full access**),
 - **PUT/GET** instructions (**remote memory access** in distributed shared-memory systems) or
 - **SEND/RECV** instructions (memory-passing in distributed memory systems where **no access**)
- Access cost to this **memory address space** is **non-uniform** (10-50 usec) than to local memory (2-10 nsec)
- If local caches & coherency are maintained, then may also be called **Cache Coherent NUMA (CC-NUMA)**
 - ⇒ Access cost can be partially controlled by programmers through data locality and thread placement

A. Memory Architectures

Note on **Non**-Uniform Memory Access (NUMA)

Uniform Memory Access (UMA) architectures

- There is a single **unified memory address space (UAS)** visible to all CPUs
- Access to this **memory address space** is via **LOAD** and **STORE** instructions
- Access cost to this **memory address space** is **equal** (uniform) to the access cost to any local memory (access to memory word in local memory/cache ~**2-10 nsec**) $\Rightarrow$ UMA
 - Hardware ensure cache coherency across processors $\Rightarrow$ Access cost is uniform and generally invisible to programmers

Non-uniform Memory Access (**N**UMA) architectures

- There is a single **memory address space** that may be partially visible to all CPUs (dep. on hw. topology)
- Access to this **memory address space** is via
 - **LOAD/STORE** (**shared-memory** operations where **full access**),
 - **PUT/GET** instructions (**remote memory access** in **distributed shared-memory** systems) or
 - **SEND/RECV** instructions (memory-passing in **distributed memory** systems where **no access**)
- Access cost to this **memory address space** is **non-uniform** (10-50 usec) than to local memory (2-10 nsec)
- If local caches & coherency are maintained, then may also be called **Cache Coherent NUMA (CC-NUMA)** $\Rightarrow$ Access cost can be partially controlled by programmers through data locality and thread placement

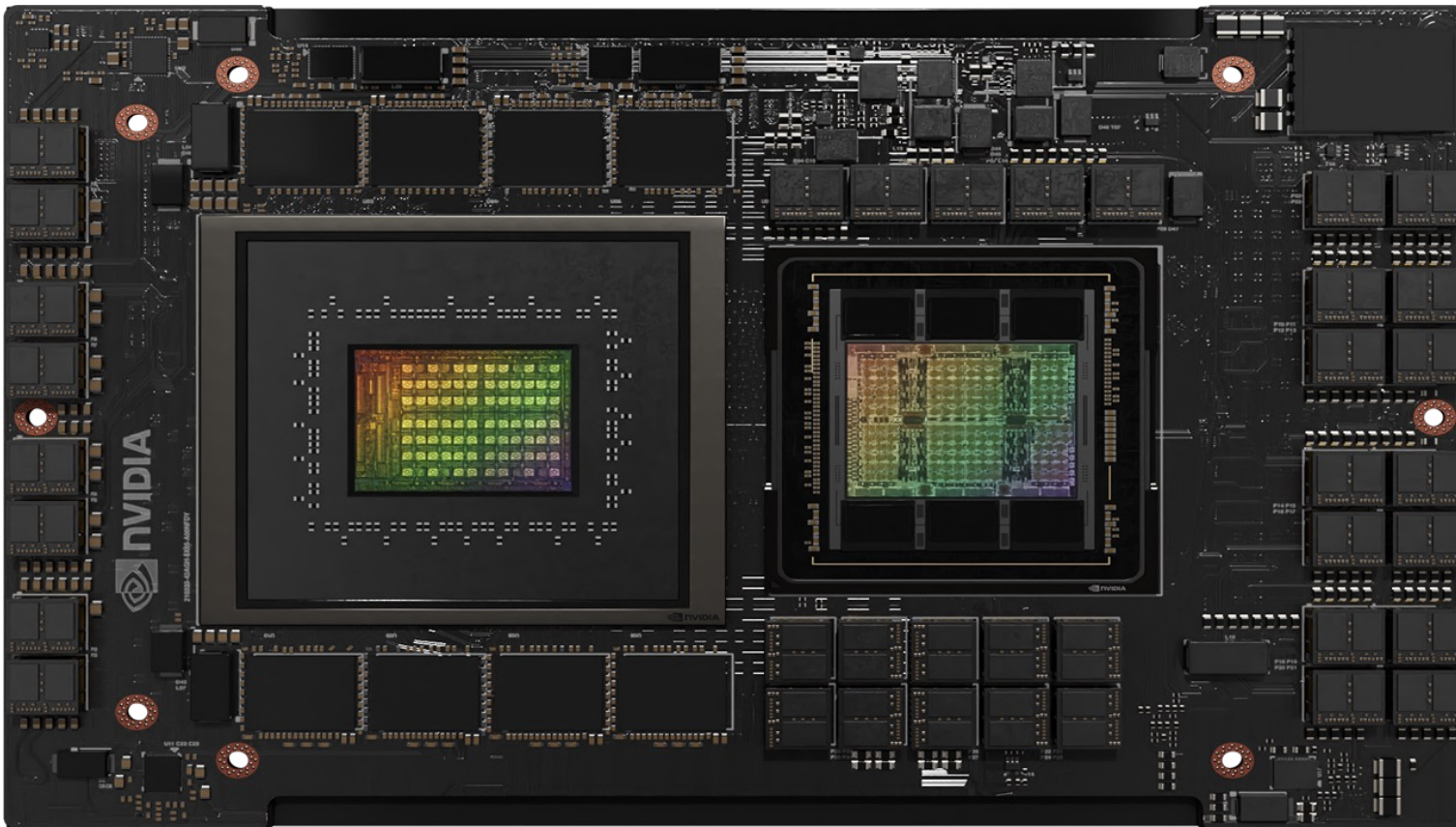
A. Memory Architectures

Note on Uniform (UMA) and Non-Uniform Memory Access (NUMA)

Memory Architecture	Uniform Memory Access (UMA)	Non-uniform Memory Access (NUMA)
Memory address space	Unified, same for all CPUs (shared)	Shared but physically distributed
Access method	LOAD/STORE instructions	LOAD/STORE, PUT/GET, SEND/RECV (depends on memory location)
Access time	Uniform	Non-uniform (depends on memory location)
Typical access latency	~ 2-10 ns	~ 10-50 μ s (remote)
Cache coherence	Hardware-maintained	Often hardware-assisted (CC-NUMA)
Programmer control	Minimal	Through data locality awareness

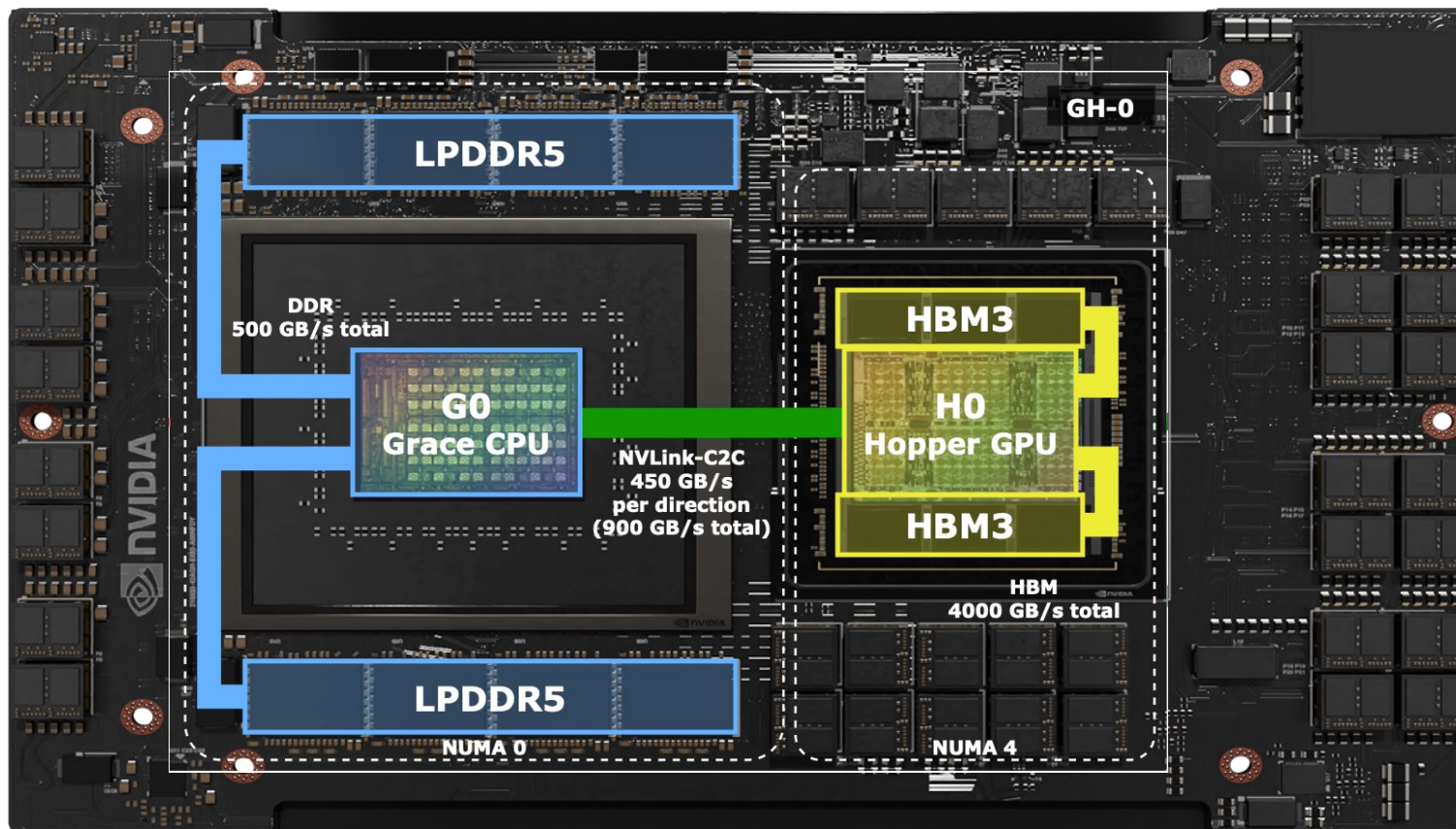
A. Memory Architectures

Famous/Recent NUMA Example: Nvidia's Grace Hopper Superchip



A. Memory Architectures

Famous/Recent NUMA Example: Nvidia's Grace Hopper Superchip



Grace: HPC-oriented 72× Arm Neoverse V2 CPU cores, 64-bit data center architecture

Memory: Up to 480GB of LPDDR5 ECC, up to 500GB/s
L1i 64 KB, L1d 64 KB / core
L2d 1 MB shared b/w cores
L3d 114 MB shared b/w cores

Hopper: 132 streaming multiprocessors (SMs)

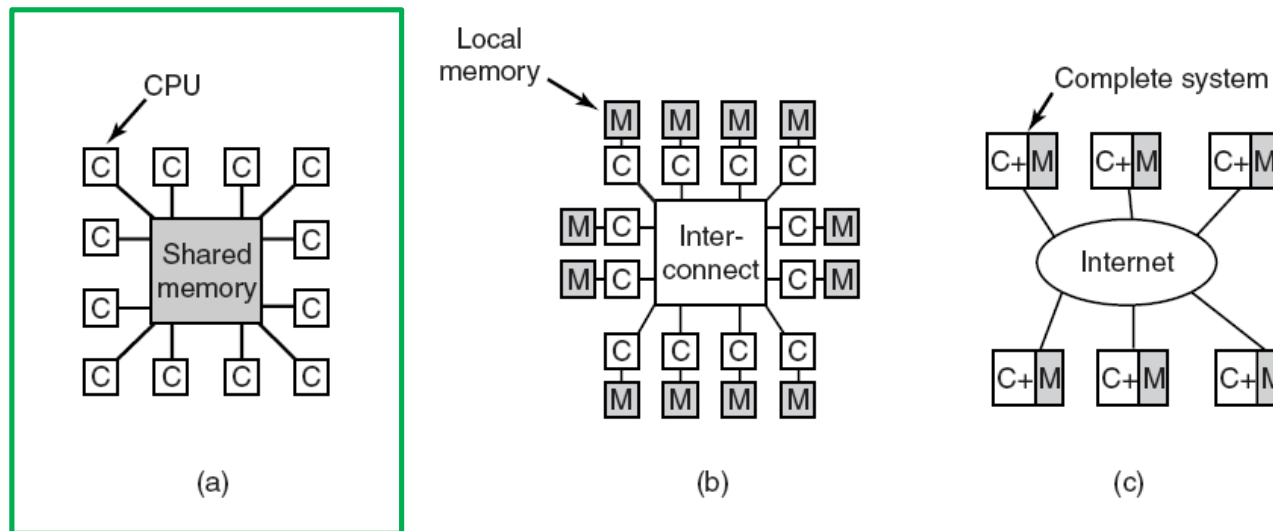
Memory:
96 GB HBM3, 4TB/s
128 GB LPDDR5 RAM
L1 256 KB private / SM
L2 52 MB shared between SMs

Up to 2048 concurrent threads

Parallel Computers and Their Programming

As Easy as A, B, C

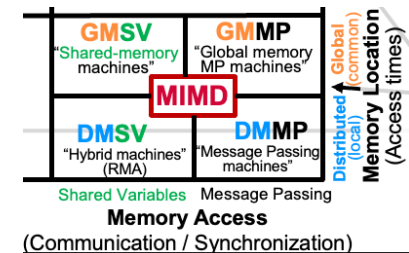
A. Memory architectures



B. Programming models

C. Network topologies

- (a) A **shared-memory multiprocessor**: **common memory** + **full access**
- (b) A parallel distributed-memory (message-passing) multicomputer
- (c) A wide area distributed multicomputer system



Parallel Computers and Their Programming

As Easy as A, B, C

A. Memory architectures

Machine model 1: Shared-memory

⇒ Symmetric multiprocessing & UMA

Machine model 2: Distributed-memory

⇒ Asymmetric multiprocessing & NUMA

Machine model 3: Hybrid distributed shared-memory (combines 1+2)

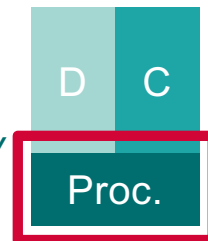
⇒ Asymmetric multiprocessing & NUMA, GAS

Machine model 4: Distributed shared-memory (variation of 3)

⇒ Asymmetric multiprocessing & NUMA, DGAS

Machine model 5: Fully distributed systems (not parallel, not covered by)

What the machine offers
w.r.t. memory



B. Parallel programming models

C. Network topologies

Machine Model 1: (Centralized) Shared-Memory Symmetric Multiprocessing (SMP)

Processors (multicore, manycore) all connected to a large **common** (global) memory with a **full** access to a **unified** address space (**UAS**); each processor **may** have a local cache and a network interface card (**NIC**)

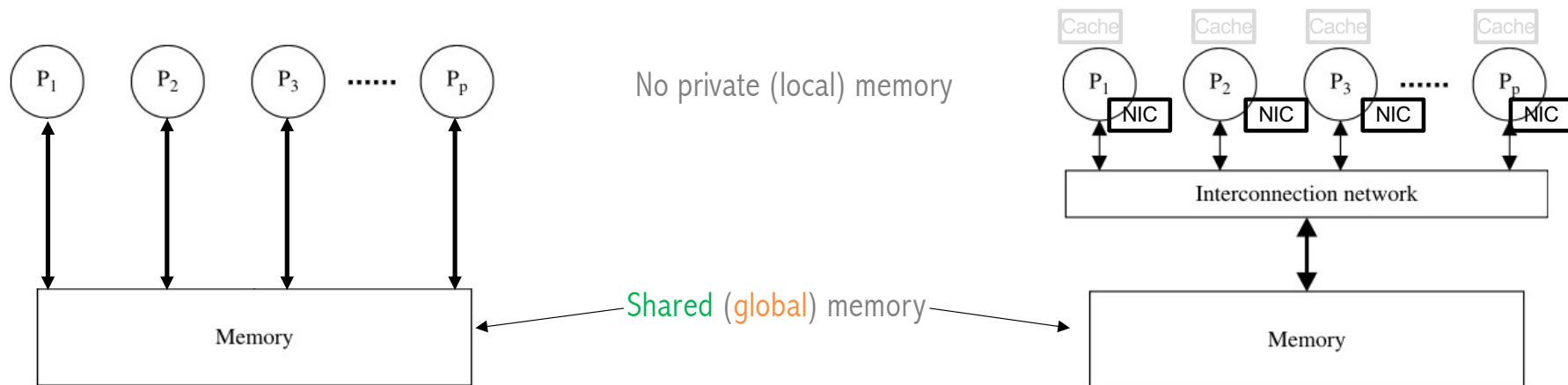


- Can perform **symmetric** multiprocessing (**SMP**) due to **uniform** memory access (**UMA**)

Advantage: **UAS+UMA** in which all data can be found/accessed

Cost: Much cheaper to access data in any local memory/cache (if present → see later in bus-based systems) than main **shared** memory

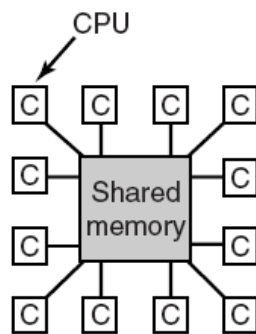
Limitation: Scaling depends on **performance of interconnection network** (*bus or LAN*), as global tasks executed in parallel require **synchronization** (contention!) and **data exchange** (cost!) over the network



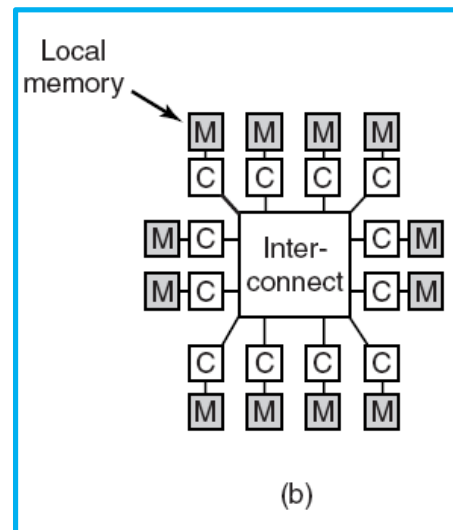
Parallel Computers and Their Programming

As Easy as A, B, C

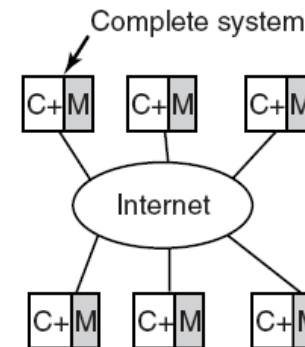
A. Memory architectures



(a)



(b)



(c)

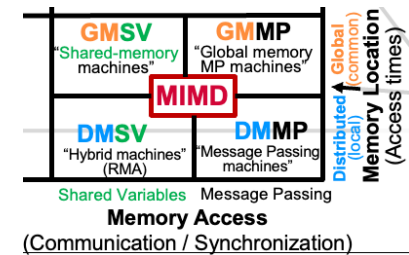
B. Programming models

C. Network topologies

(a) A shared-memory multiprocessor

(b) A **parallel distributed-memory** (message-passing) **multicomputer**:
not common memory + no full access

(c) A wide area distributed multicomputer system



Parallel Computers and Their Programming

As Easy as A, B, C

A. Memory architectures

Machine model 1: Shared-memory

⇒ Symmetric multiprocessing & UMA

Machine model 2: Distributed-memory

⇒ Asymmetric multiprocessing & NUMA

Machine model 3: Hybrid distributed shared-memory (combines 1+2)

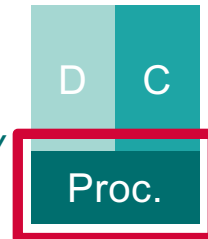
⇒ Asymmetric multiprocessing & NUMA, GAS

Machine model 4: Distributed shared-memory (variation of 3)

⇒ Asymmetric multiprocessing & NUMA, DGAS

Machine model 5: Fully distributed systems (not parallel, not covered by)

What the machine offers
w.r.t. memory



B. Parallel programming models

C. Network topologies

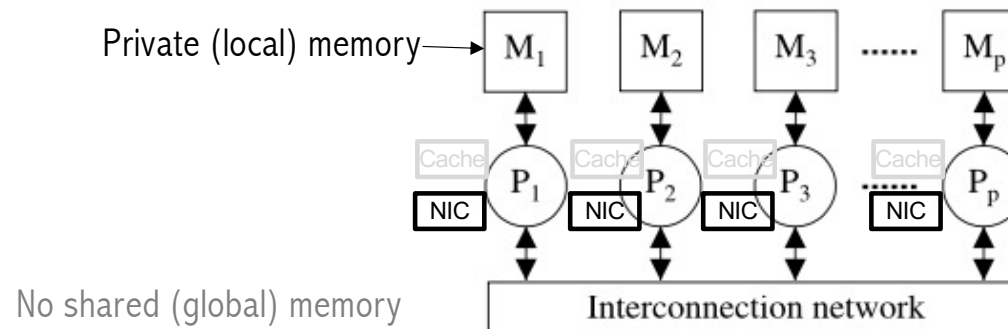
Machine Model 2: Distributed-Memory Asymmetric Multiprocessing (AMP)

- Each processor has its **own memory** (perhaps also own cache)
- **NO** (direct) access (nothing shared) to another processor's memory (**NORMA**)
 - ⇒ **Message-passing required** (master-worker aka *asymmetric multiprocessing AMP*)
 - ⇒ With good interconnect, a short message can be sent in **10-50 usec** => cost not in the hands of the programmer
- Each processor has a **NIC** (network interface card) for communication and synchronization
 - ⇒ Access to private (local) memory goes through processors (vs. directly via network as in shared global mem. case)
 - ⇒ **NUMA** characteristics: accessing local memory (still) faster than “accessing” remote memory

Advantage: Memory is *scalable*; *Excludes* race conditions; *Fast local* memory access

Cost: **NUMA** costs; network interconnectivity (cables, switches, blocking factor – later slides)

Challenge: Forces the programmer to **explicitly** think about **data distribution** across the local memories; process **synchronization**, **communication**, and **deadlocks**



Parallel Computers and Their Programming

As Easy as A, B, C

A. Memory architectures

Machine model 1: Shared-memory

⇒ Symmetric multiprocessing & UMA

Machine model 2: Distributed-memory

⇒ Asymmetric multiprocessing & NUMA

Machine model 3: Hybrid distributed shared-memory (combines 1+2)

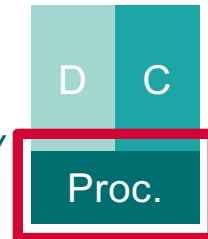
⇒ Asymmetric multiprocessing & NUMA, GAS

Machine model 4: Distributed shared-memory (variation of 3)

⇒ Asymmetric multiprocessing & NUMA, DGAS

Machine model 5: Fully distributed systems (not parallel, not covered by)

What the machine offers
w.r.t. memory



B. Parallel programming models

C. Network topologies

Machine Model 3: Hybrid Distributed Shared-Memory

Global Address Space (GAS) | A/symmetric Multiprocessing

Memory is locally **shared** but physically **distributed** ⇒ **most computers today**

- Any local processor/device can *locally* access any address in its **own** memory and in the memory shared locally with others (e.g., multicore CPUs)
- **Message passing** is required for processors that **cannot directly access** another processor's memory (e.g., other CPUs and GPUs on node and across nodes)



- Each processor has a **NIC** for all communication and synchronization
- **NUMA** characteristics: accessing local memory (still) faster than “accessing” remote memory at non-uniform cost

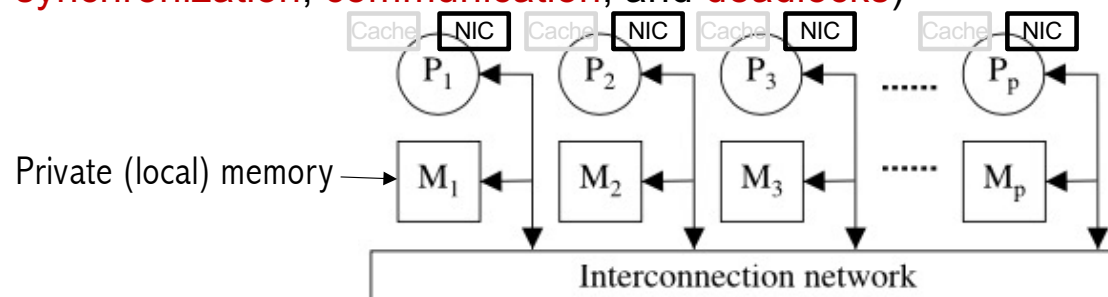
Advantages: Increased **scalability**, **fast** & **UMA** for local memory, no races across memories

⇒ **Easier to design** by integrating **shared-** with **distributed-**memory architectures

⇒ Advantages common to both **shared-** & **distributed-**memory architectures

Cost: **NUMA** costs, network interconnectivity (cables, switches, blocking factor)

Disadvantage: Increased *programming complexity* (explicit data distribution across memories, process **synchronization**, **communication**, and **deadlocks**)



Machine Model 3: Hybrid Distributed Shared-Memory

Note: How to Program Clusters of SMPs (=AMPs)?

Here: for reference only, parallel programming models described later (Chapter 5.3)

1. Treat machine as “**flat**”: use **message-passing** everywhere, even within an SMP node of the cluster (simple, but ignores an important part of *local* memory hierarchy that can be exploited for fast UMA)

Note: See programming model 3 in later slides

or

2. **Hybrid**: Use **shared-memory programming** (with threads) within one SMP and message passing outside of an SMP (across **distributed memories**) => most common!

Note: See programming models 2+3 or 5 in later slides

⚠ The *largest* and *fastest* computers in the world today employ both **shared**-(locally) and **distributed-memory** (locally, globally) architectures

⇒  Recall the Multilevel and heterogeneous parallelism (MLHP) from Chapter 5.1b.

Parallel Computers and Their Programming

As Easy as A, B, C

A. Memory architectures

Machine model 1: Shared-memory

⇒ Symmetric multiprocessing & UMA

Machine model 2: Distributed-memory

⇒ Asymmetric multiprocessing & NUMA

Machine model 3: Hybrid distributed shared-memory (combines 1+2)

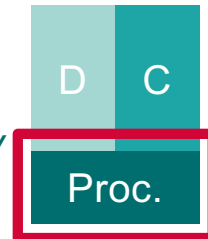
⇒ Asymmetric multiprocessing & NUMA, GAS

Machine model 4: Distributed shared-memory (variation of 3)

⇒ Asymmetric multiprocessing & NUMA, DGAS

Machine model 5: Fully distributed systems (not parallel, not covered by)

What the machine offers
w.r.t. memory



B. Parallel programming models

C. Network topologies

Machine Model 4: Distributed Shared-Memory

Distributed Global Address Space (**D**GAS), also called Partitioned Global Address Space (PGAS)

Memory is private (locally), **physically distributed** (globally) but **logically shared** (globally) (either HW/SW)

- Any processor can **logically access** any address in the **distributed global address space** (**D**GAS)
- Cache lines (or memory pages) are passed around machine over the network

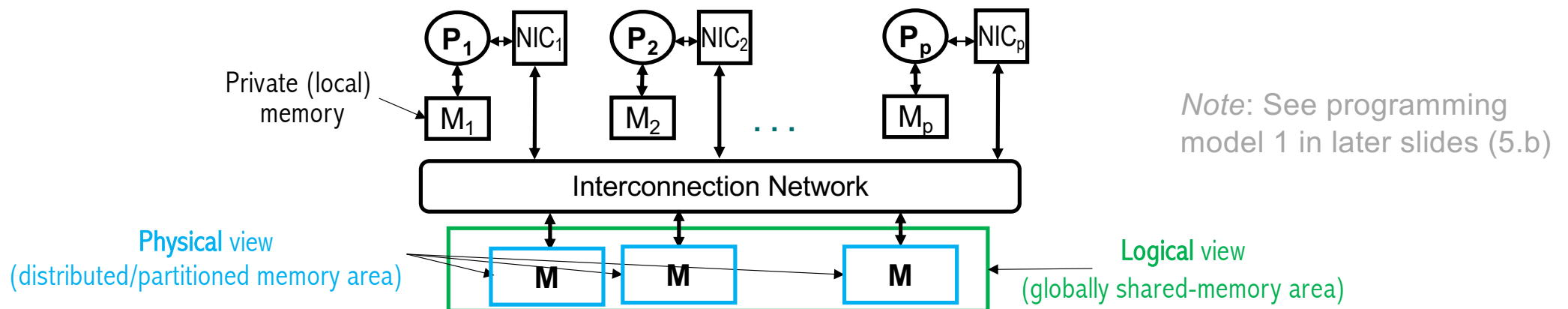
Advantages: Easier to design a machine with *memory that scales with the algorithm*

⇒ Message passing is 'hidden' from programmer ⇒ Easier to program ☺

(⚠ only hides the *mechanism* of communication, not the latency of communication)

Cost: **N**UMA & message passing costs, network interconnectivity (cables, switches, blocking)

Limitation: HW implementations need **expensive cache-coherency protocols** to keep consistent cached copies (in private memories) of the same **distributed** global address space



Machine Model 4: Distributed Shared-Memory Implementation

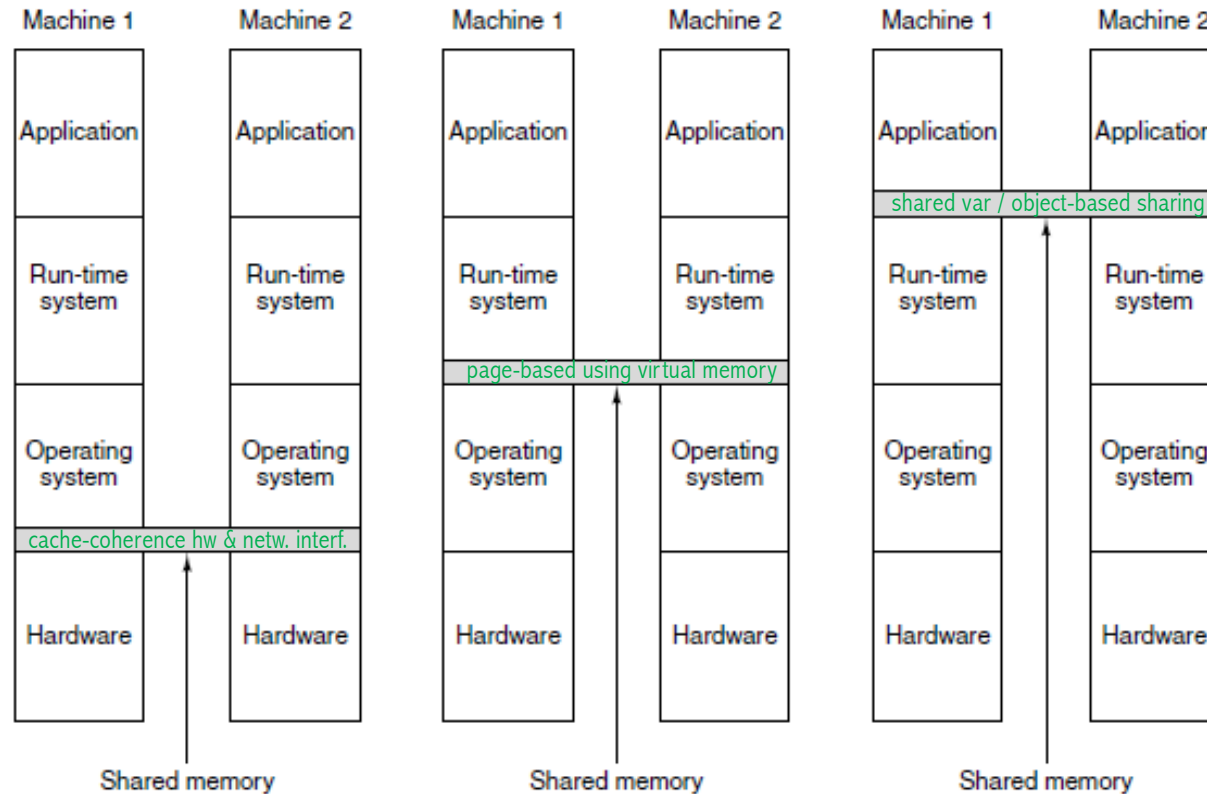
⚠️ DS-M @ HW

Most common, under HW ccNUMA:

- High performance
- Low latency
- Transparency to programmers
- Strong HW support

Examples:

- AMD EPYC + Infinity Fabric (NUMA domains)
- Intel Xeon with QPI/UPI
- IBM POWER systems
- NVIDIA Grace Hopper CPU



(a) DS-M @ hardware level (b) DS-M @ operating system (c) DS-M @ user-level software
(cache coherence circuits, bus/NICs) (page-based using virtual mem) (shared variable/object)

DS-M @ OS:

- Too slow for fine-grained sharing
- TLB & page fault overheads
- replaced by RDMA and coherent interconnects.

DS-M @ User:

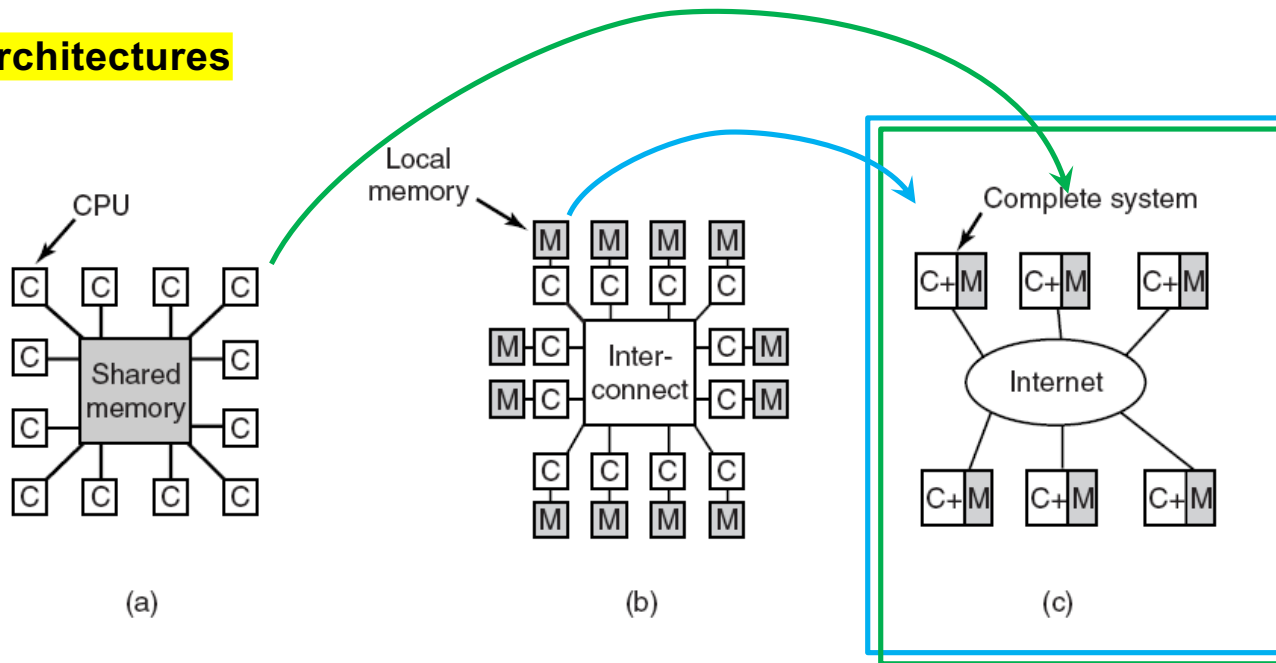
- Hard to program/debug
- performance unpredictable
- replaced by MPI, PGAS (UPC++, Chapel), and shared-nothing cluster architectures.

Figure 8-21. Various layers where distributed shared-memory can be implemented/emulated.

Parallel Computers and Their Programming

As Easy as A, B, C

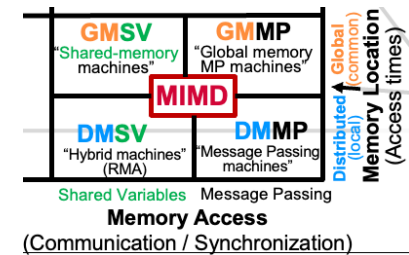
A. Memory architectures



B. Programming models

C. Network topologies

- (a) A shared-memory multiprocessor
- (b) A parallel distributed-memory (message-passing) multicomputer
- (c) A wide area **distributed multicomputer system** which typically consists of **shared-** (locally) and/or **distributed-memory** (globally) units



Parallel Computers and Their Programming

As Easy as A, B, C

A. Memory architectures

Machine model 1: Shared-memory

⇒ Symmetric multiprocessing & UMA

Machine model 2: Distributed-memory

⇒ Asymmetric multiprocessing & NUMA

Machine model 3: Hybrid distributed shared-memory (combines 1+2)

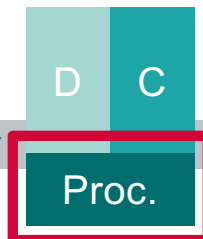
⇒ Asymmetric multiprocessing & NUMA, GAS

Machine model 4: Distributed shared-memory (variation of 3)

⇒ Asymmetric multiprocessing & NUMA, DGAS

Machine model 5: Fully distributed systems (not parallel, not covered by)

What the machine offers
w.r.t. memory

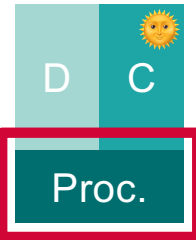


B. Parallel programming models

C. Network topologies

Machine Model 5: Fully Distributed Systems

Note: Only briefly mentioned in the Processing Context



- Connected *complete computer systems* over a **wide area network** (e.g., Internet)
- Each system has own memory and systems communicate exclusively by message passing
 - Message times are often **10-100 msec** => Access cost invisible to and out of control of programmers
 - Compare with
 - 10-50 usec: distributed-memory machine models 2-4
 - 2-10 nsec: shared-memory machine model 1

Advantages: Fault tolerance, easy to scale out

Cost: Long(er) communication delays force distributed systems to be used in *different ways* than the tightly coupled distributed-memory systems (machine models 2, 3, 4)

Challenges: Coordination, consensus => back to the “Communication” world 

📷 Big Picture of All **MIMD** Machine Models

My view of **Parallel** and/or **Distributed** Multiprocessing Systems

Item	Parallel system		
	Multiprocessor	Multicomputer	Distributed System
Node configuration	CPU	CPU, RAM, net interface	Complete computer
Node peripherals	All shared	Shared exc. maybe disk	Full set per node
Location	Same rack	Same room	Possibly worldwide
Internode communication	Shared RAM	Dedicated interconnect	Traditional network
Operating systems	One, shared	Multiple, same	Possibly all different
File systems	One, shared	One, shared	Each node has own
Administration	One organization	One organization	Many organizations
	Shared-memory		Distributed-memory
	Hybrid distributed shared-memory		

Figure 8-29. Comparison of three kinds of multiprocessing systems.

Parallel Computers and Their Programming

What We Learned So Far

A. Memory architectures

Machine model 1: Shared-memory

⇒ Symmetric multiprocessing & UMA

Machine model 2: Distributed-memory

⇒ Asymmetric multiprocessing & NUMA

Machine model 3: Hybrid distributed shared-memory (combines 1+2)

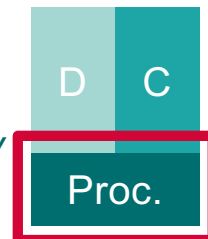
⇒ Asymmetric multiprocessing & NUMA, GAS

Machine model 4: Distributed shared-memory (variation of 3)

⇒ Asymmetric multiprocessing & NUMA, DGAS

Machine model 5: Fully distributed systems (not parallel, not covered by)

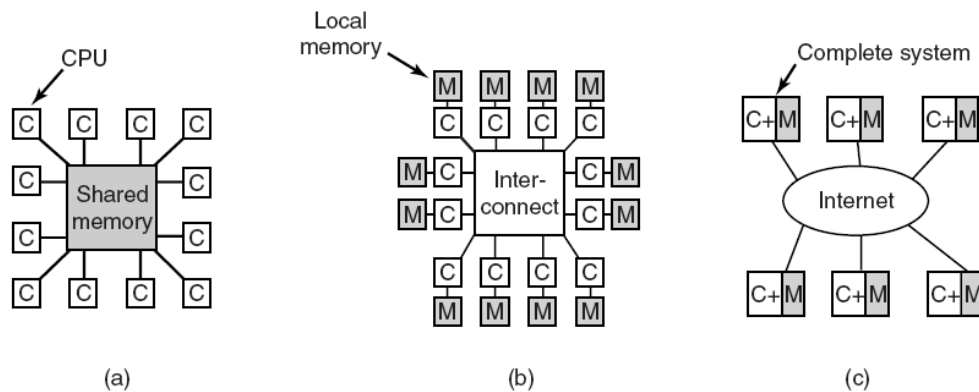
What the machine offers
w.r.t. memory



B. Parallel programming models

C. Network topologies

What Have We Learned Today?



a Memory Architectures

b Parallel Programming Models

c Network Topologies

Summary

- **Memory organization plays a critical role** in the design and organization of parallel (and distributed) **MIMD** computers
- Leads to different **machine models** (MMs)
- MM1: **Shared-memory**: **common memory** + **full access**
- MM2: **Distributed-memory**: **no** common memory + **no** access at all
- MM3: Hybrid **distributed shared-memory**: **not all common memory** + **not all full access** (combination of MM1+MM2)
- MM4: **Distributed shared-memory**: **not all common memory** + **not all full access** (variation of MM3)
- **MM5: Fully distributed systems**: **no** common memory + **no** access at all (not covered in HPC part)